



3

Building a Deep Neural Network with PyTorch

In the previous chapter, we learned how to code a neural network using PyTorch. We also learned about the various hyperparameters that are present in a neural network, such as its batch size, learning rate, and loss optimizer. In this chapter, we will shift gears and learn how to perform image classification using neural networks. Essentially, we will learn how to represent images and tweak the hyperparameters of a neural network to understand their impact.

For the sake of not introducing too much complexity and confusion, we only covered the fundamental aspects of neural networks in the previous chapter. However, there are many more inputs that we tweak in a network while training it. Typically, these inputs are known as **hyperparameters**. In contrast to the *parameters* in a neural network (which are learned during training), hyperparameters are provided by the person who builds the network. Changing different aspects of each hyperparameter is likely to affect the accuracy or speed of training a neural network. Furthermore, a few additional techniques such as scaling, batch normalization, and regularization help in improving the performance of a neural network. We will learn about these concepts throughout this chapter.

However, before we get to that, we will learn about how an image is represented: only then will we do a deep dive into the details of hyperparameters. While learning about the impact of hyperparameters, we will restrict ourselves to one dataset: Fashion MNIST (details about the dataset can be found at <https://github.com/zalandoresearch/fashion-mnist>), so that we can make a comparison of the impact of variations in various hyperparameters. Through this dataset, we will also be intro-

duced to training and validation data and why it is important to have two separate datasets. Finally, we will learn about the concept of overfitting a neural network and then understand how certain hyperparameters help us avoid overfitting.

In summary, in this chapter, we will cover the following topics:

- Representing an image
- Why leverage neural networks for image analysis?
- Preparing data for image classification
- Training a neural network
- Scaling a dataset to improve model accuracy
- Understanding the impact of varying the batch size
- Understanding the impact of varying the loss optimizer
- Understanding the impact of varying the learning rate
- Building a deeper neural network
- Understanding the impact of batch normalization
- The concept of overfitting

Let's get started!



All code in this chapter is available for reference in the `Chapter03` folder of this book's GitHub repository at <https://bit.ly/mcvc-2e>.

We have covered the impact of varying the learning rate in the associated code in the GitHub repo.

Representing an image

A digital image file (typically associated with the extension “JPEG” or “PNG”) is comprised of an array of pixels. A pixel is the smallest constituting element of an image. In a grayscale image, each pixel is a scalar (single) value between `0` and `255`: `0` is black, `255` is white, and anything in between is gray (the smaller the pixel value, the darker the pixel is).

On the other hand, the pixels in color images are three-dimensional vec-

tors that correspond to the scalar values that can be found in their red, green, and blue channels.

An image has *height* $\times$ *width* $\times$ *c* pixels, where *height* is the number of **rows** of pixels, *width* is the number of **columns** of pixels, and *c* is the number of **channels**. *c* is 3 for color images (one channel each for the *red*, *green*, and *blue* intensities of the image) and 1 for grayscale images. An example grayscale image containing 3 $\times$ 3 pixels and their corresponding scalar values is shown here:

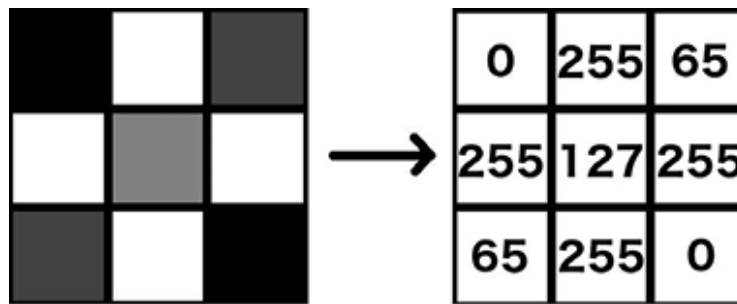


Figure 3.1: Representing an image

Again, a pixel value of 0 means that it is pitch black, while 255 means it is pure luminance (that is, pure white for grayscale and pure red/green/blue in the respective channel for a color image).

Converting images into structured arrays and scalars

Python can convert images into structured arrays and scalars as follows:



The following code can be found in the `Inspecting_grayscale_images.ipynb` file located in the `Chapter03` folder on GitHub at <https://bit.ly/mcvp-2e>.

1. Download a sample image or upload a custom image of your own:

```
!wget https://www.dropbox.com/s/1981ee/Hemanvi.jpeg
```

2. Import the `cv2` (to read an image from disk) and `matplotlib` (to plot the loaded image) libraries, and read the downloaded image into the Python environment:

```
%matplotlib inline
import cv2, matplotlib.pyplot as plt
img = cv2.imread('Hemanvi.jpeg')
```

In the preceding line of code, we leverage the `cv2.imread` method to read the image. This converts an image into an array of pixel values.

3. We'll crop the image between the 50th and 250th rows, as well as the 40th and 240th columns. Finally, we'll convert the image into grayscale using the following code and plot it:

```
# Crop image
img = img[50:250,40:240]
# Convert image to grayscale
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
# Show image
plt.imshow(img_gray, cmap='gray')
```

The output of the preceding sequence of steps is as follows:

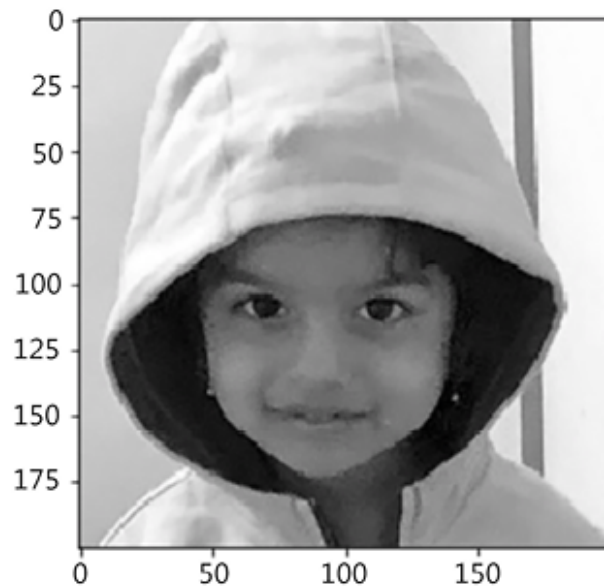


Figure 3.2: Cropped image

You might have noticed that the preceding image is represented as a 200 x 200 array of pixels. Now, let's reduce the number of pixels that are used to represent the image so that we can overlay the pixel values on the image (this would be tougher to do if we were to visualize the pixel values over a 200 x 200 array, compared to a 25 x 25 array).

4. Convert the image into a 25 x 25 array and plot it:

```
img_gray_small = cv2.resize(img_gray,(25,25))  
plt.imshow(img_gray_small, cmap='gray')
```

This results in the following output:

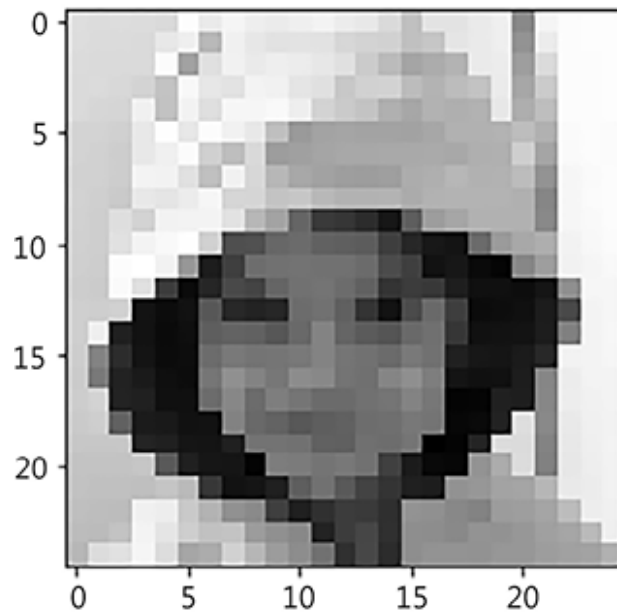


Figure 3.3: Resized image

Naturally, having fewer pixels to represent the same image results in a blurrier output.

- Let's inspect the pixel values. Note that in the following output, due to space constraints, we have pasted only the first four rows of pixel values:

```
print(img_gray_small)
```

This results in the following output:

```
array([[222, 220, 221, 220, 218, 253, 234, 245, 238, 235, 239, 243, 236,
       232, 218, 193, 228, 228, 234, 239, 139, 245, 252, 253, 253],
       [221, 219, 219, 218, 232, 239, 186, 240, 231, 226, 227, 226, 215,
       212, 209, 193, 199, 229, 234, 239, 150, 236, 252, 253, 253],
       [219, 218, 218, 218, 251, 163, 224, 241, 234, 238, 236, 231, 224,
       204, 188, 166, 173, 180, 234, 236, 159, 219, 252, 252, 253],
       [218, 219, 216, 211, 196, 248, 231, 228, 243, 241, 229, 224, 201,
       209, 210, 189, 181, 189, 196, 235, 168, 204, 252, 252, 253],
```

Figure 3.4: Pixel values of input image

The same set of pixel values, when copied and pasted into MS Excel and color-coded by pixel value, would look as follows:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y
1	222	220	221	220	218	253	234	245	238	235	239	243	236	232	218	193	228	228	234	239	139	245	252	253	253
2	221	219	219	218	232	239	186	240	231	226	227	226	215	212	209	193	199	229	234	239	150	236	252	253	253
3	219	218	218	218	251	163	224	241	234	238	236	231	224	204	188	166	173	180	234	236	159	219	252	252	253
4	218	219	216	211	196	248	231	228	243	241	229	224	201	209	210	189	181	189	196	235	168	204	252	252	253
5	218	214	213	240	195	242	223	246	246	249	238	211	203	196	177	168	179	176	179	231	175	191	252	252	253
6	212	212	208	232	254	232	252	241	232	192	155	164	166	165	164	163	168	178	178	181	190	178	250	252	251
7	211	209	205	232	240	251	208	191	217	158	161	166	169	169	170	170	171	169	176	177	206	166	250	252	251
8	209	209	205	243	242	225	193	241	215	184	169	163	159	158	160	173	176	184	178	179	189	150	246	250	252
9	210	207	203	232	229	236	246	214	213	196	199	185	179	179	181	172	179	180	180	181	177	136	246	251	252
10	209	206	222	212	242	243	244	226	184	165	104	61	57	48	27	97	158	167	178	178	178	139	246	249	252
11	208	206	225	243	249	254	209	82	85	105	109	100	98	95	65	43	28	24	109	156	169	175	242	248	251
12	208	205	252	255	242	153	33	66	111	116	117	116	115	109	78	66	22	27	14	9	137	159	241	245	249
13	205	204	250	225	63	15	42	77	71	104	115	118	110	101	56	64	60	34	20	20	17	25	145	246	246
14	208	206	209	23	16	22	90	45	39	43	110	115	99	56	23	78	107	65	15	17	20	32	76	244	246
15	208	239	37	22	14	19	97	102	100	90	108	133	104	94	88	108	114	57	21	22	23	33	130	243	246
16	205	133	48	24	15	17	110	124	118	119	124	134	119	116	109	123	116	36	27	25	31	44	242	243	246
17	204	124	38	30	19	16	120	146	133	121	142	138	118	114	135	145	128	24	20	21	33	136	236	243	244
18	205	212	39	37	20	20	110	137	110	128	119	109	109	115	119	133	121	8	9	36	34	137	237	241	243
19	204	206	101	31	29	21	22	132	108	102	91	97	101	111	113	122	118	11	14	38	222	139	233	240	242
20	200	200	200	41	36	25	24	37	117	117	116	101	99	114	111	119	4	8	41	220	219	134	232	239	244
21	197	196	196	199	92	37	26	25	8	127	125	118	122	116	67	31	13	11	150	173	220	131	231	238	242
22	195	193	193	192	198	187	58	22	25	37	97	115	93	70	55	36	33	148	153	165	166	183	233	236	242
23	192	190	189	240	237	202	180	147	140	66	36	52	64	61	51	150	146	138	134	157	159	166	189	237	240
24	189	188	197	244	229	206	194	196	157	146	138	39	63	73	53	144	143	139	150	148	153	161	167	187	239
25	184	220	225	245	221	167	173	209	183	157	143	116	53	74	49	144	150	150	148	153	153	158	162	165	178

Figure 3.5: Pixel values corresponding to the image

As we mentioned previously, the pixels with a scalar value closer to 255 appear lighter, while those closer to 0 appear darker.

Creating a structured array for colored images

The preceding steps apply to color images too, which are represented as three-dimensional vectors. The brightest red pixel is denoted as $(255, 0, 0)$. Similarly, a pure white pixel in a three-dimensional vector

image is represented as `(255, 255, 255)`. With this in mind, let's create a structured array of pixel values for a colored image:



The following code can be found in the `Inspecting_color_images.ipynb` file located in the `Chapter03` folder on GitHub at <https://bit.ly/mcvp-2e>.

1. Download a color image:

```
!wget https://www.dropbox.com/s/l98lee/Hemanvi.jpeg
```

2. Import the relevant packages and load the image:

```
import cv2, matplotlib.pyplot as plt
%matplotlib inline
img = cv2.imread('Hemanvi.jpeg')
```

3. Crop the image:

```
img = img[50:250, 40:240, :]
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Note that in the preceding code, we've reordered the channels using the `cv2.cvtColor` method. We've done this because when we import images using `cv2`, the channels are ordered as blue first, green next, and finally, red; typically, we are used to looking at images in RGB channels, where the sequence is red, green, and then blue.

4. Plot the image that's obtained:

```
plt.imshow(img)
print(img.shape)
# (200, 200, 3)
```

This results in the following output (note that if you are reading the print book and haven't downloaded the color image bundle, the following image will appear in grayscale):

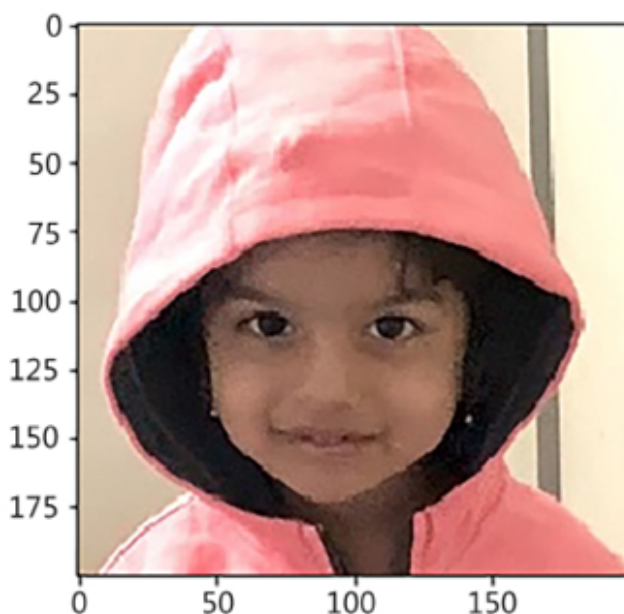


Figure 3.6: Image in the RGB format

5. The bottom-right 3 x 3 array of pixels can be obtained as follows:

```
crop = img[-3:,-3:]
```

6. Print and plot the pixel values:

```
print(crop)
plt.imshow(crop)
```

The preceding code results in the following output:

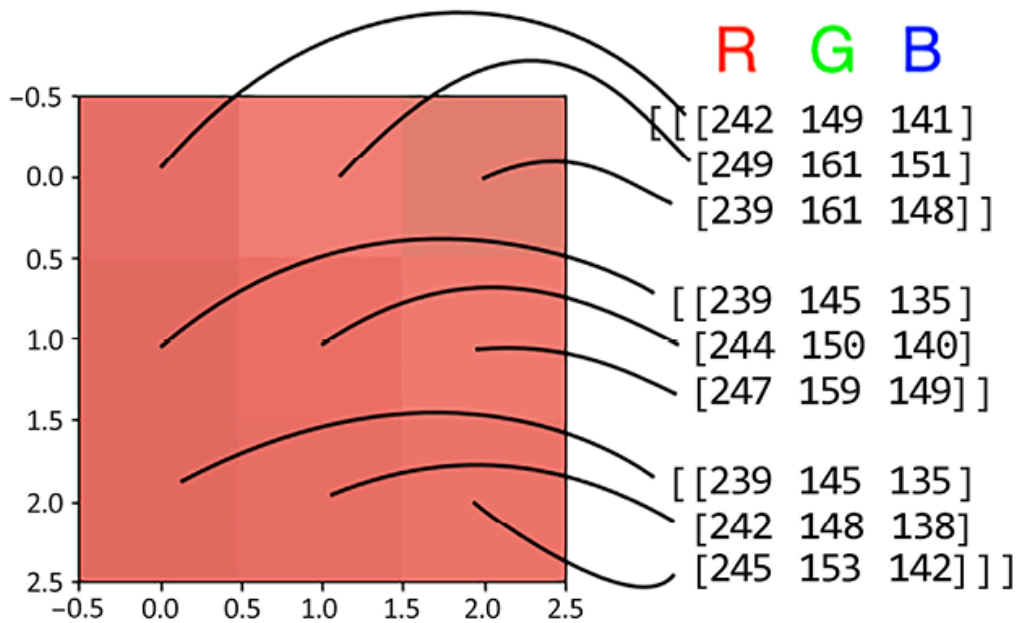


Figure 3.7: RGB values of a patch of the image

Now that we have learned how to represent an image (i.e., a file on your computer) as a tensor, we are now in a position to learn various mathematical operations and techniques that can leverage these tensors to perform tasks, such as image classification, object detection, image segmentation and many more throughout this book.

But first, let's understand why **artificial neural networks** (ANNs) are useful for image analysis.

Why leverage neural networks for image analysis?

In traditional computer vision, we would create a few features for every image before using them as input. Let's look at a few such features based on the following sample image, in order to appreciate the effort we save by training a neural network:

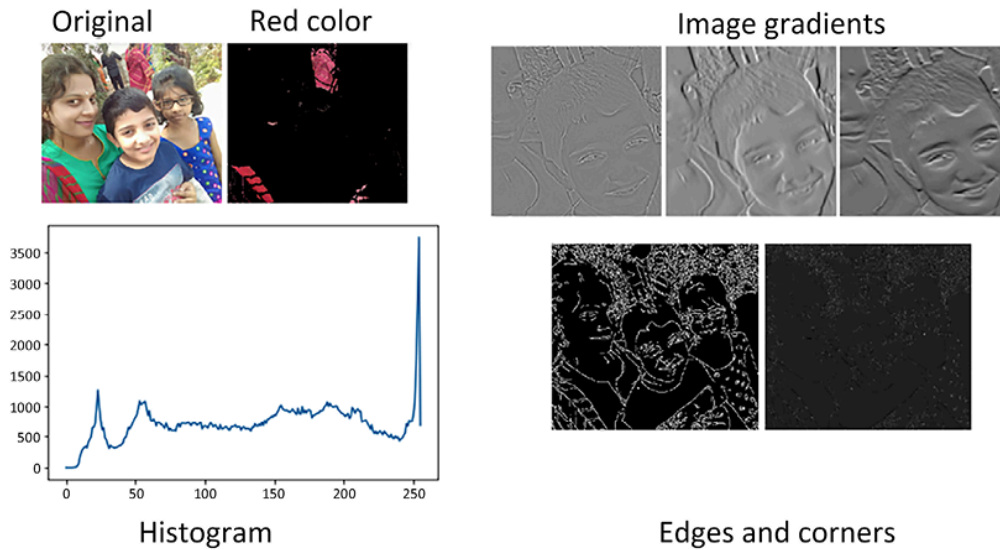


Figure 3.8: A subset of features that can be generated from an image

Note that we will not walk you through how to get these features, as the intention here is to help you realize why creating features manually is a suboptimal exercise. However, you can familiarize yourself with the different feature extraction methods at

https://docs.opencv.org/4.x/d7/da8/tutorial_table_of_content_imgproc.html:

- **Histogram feature:** For some tasks, such as auto-brightness or night vision, it is important to understand the illumination in the picture: that is, the fraction of pixels that are bright or dark.
- **Edges and corners feature:** For tasks such as image segmentation, where it is important to find the set of pixels corresponding to each person, it makes sense to extract the edges first because the border of a person is just a collection of edges. In other tasks, such as image registration, it is vital that key landmarks are detected. These landmarks will be a subset of all the corners in an image.
- **Color separation feature:** In tasks such as traffic light detection for a self-driving car, it is important that the system understands what color is displayed on the traffic lights.
- **Image gradients feature:** Taking the color separation feature a step further, it might be important to understand how the colors change at the pixel level. Different textures can give us different gradients,

which means they can be used as texture detectors. In fact, finding gradients is a prerequisite for edge detection.

These are just a handful of such features. There are so many more that it is difficult to cover all of them. The main drawback of creating these features is that you need to be an expert in image and signal analysis and should fully understand what features are best suited to solve a problem. Even if both constraints are satisfied, there is no guarantee that such an expert will be able to find the right combination of inputs, and even if they do, there is still no guarantee that such a combination will work in new, unseen scenarios.

Due to these drawbacks, the community has largely shifted to neural network-based models. These models not only find the right features automatically but also learn how to optimally combine them to get the job done. As we have already seen in the first chapter, neural networks act as both feature extractors and classifiers.

Now that we've had a look at some examples of historical feature extraction techniques and their drawbacks, let's learn how to train a neural network on images.

Preparing our data for image classification

Given that we are covering multiple scenarios in this chapter, in order for us to see the advantage of one scenario over the other, we will work on a single dataset throughout this chapter: the Fashion MNIST dataset: which contains images of 10 different classes of clothing (shirts, trousers, shoes, and so on). Let's prepare this dataset:



The following code can be found in the `Preparing_our_data.ipynb` file located in the `Chapter03` folder on GitHub at <https://bit.ly/mcyp-2e>.

1. Start by downloading the dataset and importing the relevant packages. The `torchvision` package contains various datasets, one of which is the `FashionMNIST` dataset, which we will work on in this chapter:

```
from torchvision import datasets
import torch
data_folder = '~/data/FMNIST' # This can be any directory
# you want to download FMNIST to
fmnist = datasets.FashionMNIST(data_folder, download=True, train=True)
```

In the preceding code, we specify the folder (`data_folder`) where we want to store the downloaded dataset. Then, we fetch the `fmnist` data from `datasets.FashionMNIST` and store it in `data_folder`. Furthermore, we specify that we only want to download the training images by specifying `train = True`.

2. Next, we must store the images that are available in `fmnist.data` as `tr_images` and the labels (targets) that are available in `fmnist.targets` as `tr_targets`:

```
tr_images = fmnist.data
tr_targets = fmnist.targets
```

3. Inspect the tensors that we are dealing with:

```
unique_values = tr_targets.unique()
print(f'tr_images & tr_targets:\n\tX -{tr_images.shape}\n\tY \
-{tr_targets.shape}\n\tY-Unique Values : {unique_values}')
print(f'TASK:\n\t{len(unique_values)} class Classification')
print(f'UNIQUE CLASSES:\n\t{fmnist.classes}')
```

The output of the preceding code is as follows:

```

tr_images & tr_targets:
  X - torch.Size([60000, 28, 28])
  Y - torch.Size([60000])
  Y - Unique Values : tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
TASK:
  10 class Classification
UNIQUE CLASSES:
  ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

```

Figure 3.9: Input and output shapes and unique classes

Here, we can see that there are 60,000 images, each 28 x 28 in size, and with 10 possible classes across all the images. Note that `tr_targets` contains the numeric values for each class, while `fmnist.classes` gives us the names that correspond to each numeric value in `tr_targets`.

4. Plot a random sample of 10 images for all the 10 possible classes:

1. Import the relevant packages in order to plot a grid of images so that you can also work on arrays:

```

import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np

```

2. Create a plot where we can show a 10 x 10 grid, where each row of the grid corresponds to a class and each column presents an example image belonging to the row's class. Loop through the unique class numbers (`label_class`) and fetch the indices of rows (`label_x_rows`) corresponding to the given class number:

```

R, C = len(tr_targets.unique()), 10
fig, ax = plt.subplots(R, C, figsize=(10,10))
for label_class, plot_row in enumerate(ax):
    label_x_rows = np.where(tr_targets == label_class)[0]

```

Note that in the preceding code, we fetch the 0th index as the output of the `np.where` condition, as it has a length of 1. It contains an array of all the indices where the target value (`tr_targets`) is equal to `label_class`.

3. Loop through 10 times to fill the columns of a given row.

Furthermore, we need to select a random value (`ix`) from the indices corresponding to a given class that were obtained previously (`label_x_rows`) and plot them:

```
for plot_cell in plot_row:
    plot_cell.grid(False); plot_cell.axis('off')
    ix = np.random.choice(label_x_rows)
    x, y = tr_images[ix], tr_targets[ix]
    plot_cell.imshow(x, cmap='gray')
plt.tight_layout()
```

This results in the following output:



Figure 3.10: Sample Fashion MNIST images

Note that in the preceding image, each row represents a sample of 10 different images all belonging to the same class.

Training a neural network

To train a neural network, we must perform the following steps:

1. Import the relevant packages
2. Build a dataset that can fetch data one data point at a time
3. Wrap the dataloader from the dataset
4. Build a model and then define the loss function and the optimizer
5. Define two functions to train and validate a batch of data, respectively
6. Define a function that will calculate the accuracy of the data
7. Perform weight updates based on each batch of data over increasing epochs

In the following lines of code, we'll perform each of the following steps:



The following code can be found in the `Steps_to_build_a_neural_network_on_FashionMNIST.ipynb` file located in the `Chapter03` folder on GitHub at <https://bit.ly/mcvp-2e>.

1. Import the relevant packages and the `fmnist` dataset:

```
from torch.utils.data import Dataset, DataLoader
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
device = "cuda" if torch.cuda.is_available() else "cpu"
from torchvision import datasets
data_folder = '~/data/FMNIST' # This can be any directory you
# want to download FMNIST to
fmnist = datasets.FashionMNIST(data_folder, download=True, train=True)
tr_images = fmnist.data
tr_targets = fmnist.targets
```

2. Build a class that fetches the dataset. Remember that it is derived from a `Dataset` class and needs three magic functions, `__init__`, `__getitem__`, and `__len__`, to **always** be defined:

```
class FMNISTDataset(Dataset):
    def __init__(self, x, y):
        x = x.float()
        x = x.view(-1, 28*28)
        self.x, self.y = x, y
    def __getitem__(self, ix):
        x, y = self.x[ix], self.y[ix]
        return x.to(device), y.to(device)
    def __len__(self):
        return len(self.x)
```

Note that in the `__init__` method, we convert the input into a floating-point number and also flatten each image into $28*28 = 784$ numeric values (where each numeric value corresponds to a pixel value). We also specify the number of data points in the `__len__` method; here, it is the length of `x`. The `__getitem__` method contains logic for what should be returned when we ask for the `ixth` data point (`ix` will be an integer between 0 and `__len__`).

3. Create a function that generates a training `DataLoader`, `trn_dl`, from the dataset called `FMNISTDataset`. This will sample 32 data points at random for the batch size:

```
def get_data():
    train = FMNISTDataset(tr_images, tr_targets)
    trn_dl = DataLoader(train, batch_size=32, shuffle=True)
    return trn_dl
```

4. Define a model, as well as the loss function and the optimizer:

```
from torch.optim import SGD
def get_model():
    model = nn.Sequential(
```

```

        nn.Linear(28 * 28, 1000),
        nn.ReLU(),
        nn.Linear(1000, 10)
    ).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = SGD(model.parameters(), lr=1e-2)
return model, loss_fn, optimizer

```

The model is a network with one hidden layer containing 1,000 neurons. The output is a 10-neuron layer, since there are 10 possible classes. Furthermore, we call the `CrossEntropyLoss` function, since the output can belong to any of the 10 classes for each image. Finally, the key aspect to note in this exercise is that we have initialized the learning rate, `lr`, to a value of `0.01` and not the default of `0.001`, to see how the model will learn for this exercise.

Note that we are not using “softmax” in the neural network at all. The range of outputs is unconstrained, in that values can have an infinite range, whereas cross-entropy loss typically expects outputs as probabilities (each row should sum to 1). Unconstrained values in output still work in this setting because `nn.CrossEntropyLoss` actually expects us to send the raw logits (that is, unconstrained values). It performs softmax internally.

5. Define a function that will train the dataset on a batch of images:

```

def train_batch(x, y, model, opt, loss_fn):
    model.train() # <- let's hold on to this until we reach
    # dropout section
    # call your model like any python function on your batch
    # of inputs
    prediction = model(x)
    # compute loss
    batch_loss = loss_fn(prediction, y)
    # based on the forward pass in `model(x)` compute all the
    # gradients of 'model.parameters()'
    batch_loss.backward()

```

```

# apply new-weights = f(old-weights, old-weight-gradients)
# where "f" is the optimizer
optimizer.step()
# Flush gradients memory for next batch of calculations
optimizer.zero_grad()
return batch_loss.item()

```

The preceding code passes the batch of images through the model in the forward pass. It also computes the loss on the batch and then passes the weights through backward propagation and updates them. Finally, it flushes the memory of the gradient so that it doesn't influence how the gradient is calculated in the next pass.

Now that we've done this, we can extract the loss value as a scalar by fetching `batch_loss.item()` on top of `batch_loss`.

6. Build a function that calculates the accuracy of a given dataset:

```

# since there's no need for updating weights,
# we might as well not compute the gradients.
# Using this '@' decorator on top of functions
# will disable gradient computation in the entire function
@torch.no_grad()
def accuracy(x, y, model):
    model.eval() # <- let's wait till we get to dropout
    # section
    # get the prediction matrix for a tensor of `x` images
    prediction = model(x)
    # compute if the location of maximum in each row
    # coincides with ground truth
    max_values, argmaxes = prediction.max(-1)
    is_correct = argmaxes == y
    return is_correct.cpu().numpy().tolist()

```

In the preceding code, we explicitly mention that we don't need to calculate the gradient by providing `@torch.no_grad()` and calculating the `prediction` values, by feed-forwarding input through the model. Next, we invoke `prediction.max(-1)` to identify the `argmax` index corre-

sponding to each row. We then compare our `argmaxes` with the ground truth through `argmaxes == y` so that we can check whether each row is predicted correctly. Finally, we return the list of `is_correct` objects after moving it to a CPU and converting it into a NumPy array.

7. Train the neural network using the following lines of code:

1. Initialize the model, loss, optimizer, and DataLoaders:

```
trn_dl = get_data()
model, loss_fn, optimizer = get_model()
```

2. Initialize the lists that will contain the accuracy and loss values at the end of each epoch:

```
losses, accuracies = [], []
```

3. Define the number of epochs:

```
for epoch in range(5):
    print(epoch)
```

4. Initialize the lists that will contain the accuracy and loss values corresponding to each batch within an epoch:

```
epoch_losses, epoch_accuracies = [], []
```

5. Create batches of training data by iterating through the DataLoader:

```
for ix, batch in enumerate(iter(trn_dl)):
    x, y = batch
```

6. Train the batch using the `train_batch` function, and store the loss value at the end of training on top of the batch as `batch_loss`.

Furthermore, store the loss values across batches in the `epoch_losses` list:

```
batch_loss = train_batch(x, y, model, optimizer, loss_fn)
epoch_losses.append(batch_loss)
```

7. We store the mean loss value across all batches within an epoch:

```
epoch_loss = np.array(epoch_losses).mean()
```

8. Next, we calculate the accuracy of the prediction at the end of training on all batches:

```
for ix, batch in enumerate(iter(trn_dl)):
    x, y = batch
    is_correct = accuracy(x, y, model)
    epoch_accuracies.extend(is_correct)
epoch_accuracy = np.mean(epoch_accuracies)
```

9. Store the loss and accuracy values at the end of each epoch in a list:

```
losses.append(epoch_loss)
accuracies.append(epoch_accuracy)
```

10. The variation of the training loss and accuracy over increasing epochs can be displayed using the following code:

```
epochs = np.arange(5)+1
plt.figure(figsize=(20,5))
plt.subplot(121)
plt.title('Loss value over increasing epochs')
plt.plot(epochs, losses, label='Training Loss')
plt.legend()
plt.subplot(122)
plt.title('Accuracy value over increasing epochs')
```

```
plt.plot(epochs, accuracies, label='Training Accuracy')
plt.gca().set_yticklabels(['{:.0f}%'.format(x*100) \
                           for x in \ plt.gca().get_yticks()])
plt.legend()
```

The output of the preceding code is as follows:

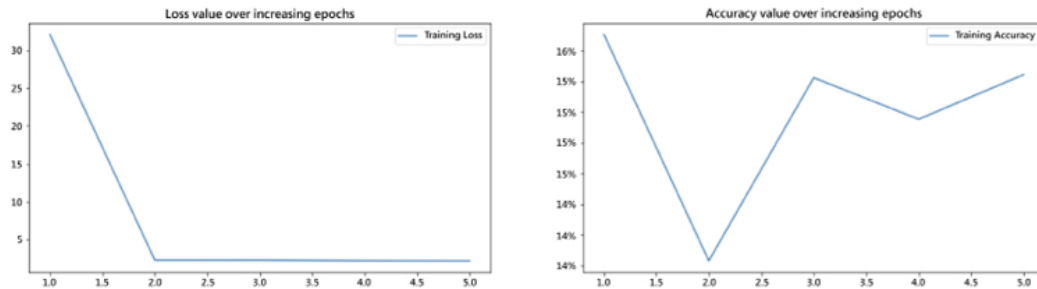


Figure 3.11: Training loss and accuracy values over increasing epochs

Our training accuracy is at 12% at the end of the five epochs. Note that the loss value did not decrease considerably over an increasing number of epochs. In other words, no matter how long we wait, it is unlikely that the model is going to provide high accuracy (say, above 80%). This calls for us to understand how the various hyperparameters that were used impact the accuracy of our neural network.

Note that since we did not specify `torch.random_seed(0)` at the start of code, the results might vary when you execute the code provided. However, the results you get should also let you reach a similar conclusion.

Now that you have a complete picture of how to train a neural network, let's study some good practices we should follow to achieve good model performance and the reasons behind using them. This can be achieved by fine-tuning various hyperparameters, some of which we will look at in the upcoming sections.

Scaling a dataset to improve model accuracy

Scaling a dataset is the process of ensuring that the variables are confined to a finite range. In this section, we will confine the independent variables' values to between 0 and 1 by dividing each input value by the maximum possible value in the dataset. This is a value of 255, which corresponds to white pixels:



For brevity's sake, we have only provided the modified code (from the previous section) in the upcoming code. The full code can be found in the `Scaling_the_dataset.ipynb` file located in the `Chapter03` folder on GitHub at <https://bit.ly/mcvp-2e>.

1. Fetch the dataset, as well as the training images and targets, as we did in the previous section.
2. Modify `FMNISTDataset`, which fetches data, so that the input image is divided by 255 (the maximum intensity/value of a pixel):

```
class FMNISTDataset(Dataset):
    def __init__(self, x, y):
        x = x.float()/255
        x = x.view(-1,28*28)
        self.x, self.y = x, y
    def __getitem__(self, ix):
        x, y = self.x[ix], self.y[ix]
        return x.to(device), y.to(device)
    def __len__(self):
        return len(self.x)
```

Note that the only change we've made here compared to the previous section is that we divide the input data by the maximum possible pixel value: 255.

Given that the pixel values range between 0 to 255, dividing them by 255 will result in values that are always between 0 and 1.

3. Train a model, just like we did in steps 4, 5, 6, and 7, of the previous section. The variations for the training loss and accuracy values are as follows:

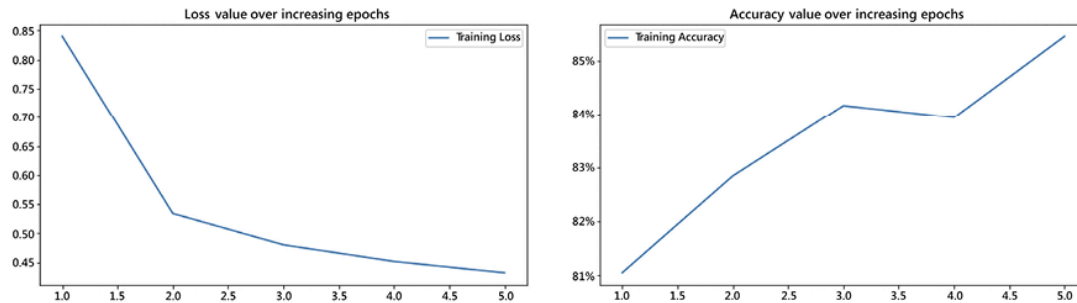


Figure 3.12: Training loss and accuracy values over increasing epochs on a scaled dataset

As we can see, the training loss consistently reduced and the training accuracy consistently increased to an accuracy of ~85%. Contrast the preceding output with the scenario where input data is not scaled, where training loss did not reduce consistently, and the accuracy of the training dataset at the end of five epochs was only 12%.

Let's dive into the possible reason why scaling helps here. We'll take the example of how a sigmoid value is calculated:

$$\text{Sigmoid} = 1 / (1 + e^{-(\text{Input} * \text{Weight})})$$

In the following table, we've calculated the **Sigmoid** column based on the preceding formula:

Input	Weight	Sigmoid
255	0.00001	0.501
255	0.0001	0.506
255	0.001	0.563
255	0.01	0.928
255	0.1	1.000
255	0.2	1.000
255	0.3	1.000
255	0.4	1.000
255	0.5	1.000
255	0.6	1.000
255	0.7	1.000
255	0.8	1.000
255	0.9	1.000
255	1	1.000

Input	Weight	Sigmoid
1	0.00001	0.500
1	0.0001	0.500
1	0.001	0.500
1	0.01	0.502
1	0.1	0.525
1	0.2	0.550
1	0.3	0.574
1	0.4	0.599
1	0.5	0.622
1	0.6	0.646
1	0.7	0.668
1	0.8	0.690
1	0.9	0.711
1	1	0.731

Figure 3.13: Sigmoid value for the different values of input and weight

In the left-hand table, we can see that when the weight values are more than 0.1, the sigmoid value does not vary with an increasing (changing) weight value. Furthermore, the sigmoid value changed only by a little when the weight was extremely small; the only way to vary the sigmoid value is by changing the weight by a very, very small amount.

However, the sigmoid value changed considerably in the right-hand table when the input value was small.

The reason for this is that the exponential of a large negative value (resulting from multiplying the weight value by a large number) is very close to 0, while the exponential value varies when the weight is multiplied by a scaled input, as seen in the right-hand table.

Now that we understand that the sigmoid value does not change considerably unless the weight values are very small, we will learn about how weight values can be influenced toward an optimal value.



Scaling the input dataset so that it contains a much smaller range of values generally helps to achieve better model accuracy.

Next, we'll learn about the impact of one of the other major hyperparameters of any neural network: **batch size**.

Understanding the impact of varying the batch size

In the previous sections, 32 data points were considered per batch in the training dataset. This resulted in a greater number of weight updates per epoch, as there were 1,875 weight updates per epoch ($60,000/32$ is nearly equal to 1,875, where 60,000 is the number of training images).

Furthermore, we did not consider the model's performance on an unseen dataset (validation dataset). We will explore this in this section.

In this section, we will compare the following:

- The loss and accuracy values of the training and validation data when the training batch size is 32
- The loss and accuracy values of the training and validation data when the training batch size is 10,000

Now that we have brought validation data into the picture, let's rerun the code provided in the *Building a neural network* section with additional code to generate validation data, as well as to calculate the loss and accuracy values of the validation dataset.



For brevity's sake, we have only provided the modified code (from the previous section) in the upcoming section. The full code can be found in the `Varying_batch_size.ipynb` file in the `Chapter03` folder in the GitHub repository at <https://bit.ly/mcnp-2e>.

Batch size of 32

Given that we have already built a model that uses a batch size of 32 during training in the previous section, we will elaborate on the additional

code that is used to work on the validation dataset. We'll skip going through the details of training the model, since this is already covered in the *Building a neural network* section. Let's get started:

1. Download and import the training images and targets.
2. In a similar manner to training images, we must download and import the validation dataset by specifying `train = False`, while calling the `FashionMNIST` method in our datasets:

```
val_fmnist = datasets.FashionMNIST(data_folder, download=True, train=False)
val_images = val_fmnist.data
val_targets = val_fmnist.targets
```

3. Import the relevant packages and define `device`.
4. Define the dataset class (`FashionMNIST`) and the functions that will be used to train on a batch of data (`train_batch`), calculate the accuracy (`accuracy`), and then define the model architecture, the loss function, and the optimizer (`get_model`).
5. Define a function that will get data: that is, `get_data`. This function will return the training data with a batch size of `32` and the validation dataset with a batch size that's the length of the validation data (we will not use the validation data to train the model; we will only use it to understand the model's accuracy on unseen data):

```
def get_data():
    train = FMNISTDataset(tr_images, tr_targets)
    trn_dl = DataLoader(train, batch_size=32, shuffle=True)
    val = FMNISTDataset(val_images, val_targets)
    val_dl = DataLoader(val, batch_size=len(val_images),
                        shuffle=False)

    return trn_dl, val_dl
```

In the preceding code, we created an object of the `FMNISTDataset` class named `val`, in addition to the `train` object that we saw earlier. Furthermore, the `DataLoader` for validation (`val_dl`) was fetched with a batch size of `len(val_images)`, while the batch size of `trn_dl` is `32`.

This is because the training data is used to train the model while we fetch the accuracy and loss metrics of the validation data. In this section and the next, we are trying to understand the impact of varying `batch_size`, based on the model's training time and accuracy.

6. Define a function that calculates the loss of the validation data: that is, `val_loss`. Note that we are calculating this separately, since the loss of training data is calculated while training the model:

```
@torch.no_grad()
def val_loss(x, y, model):
    model.eval()
    prediction = model(x)
    val_loss = loss_fn(prediction, y)
    return val_loss.item()
```

As you can see, we apply `torch.no_grad` because we don't train the model and only fetch predictions. Furthermore, we pass our `prediction` through the loss function (`loss_fn`) and return the loss value (`val_loss.item()`).

7. Fetch the training and validation DataLoaders. Also, initialize the model, loss function, and optimizer:

```
trn_dl, val_dl = get_data()
model, loss_fn, optimizer = get_model()
```

8. Train the model, as follows:

1. Initialize the lists that contain training and validation accuracy, as well as loss values over increasing epochs:

```
train_losses, train_accuracies = [], []
val_losses, val_accuracies = [], []
```

2. Loop through five epochs, and initialize lists that contain accuracy and loss across batches of training data within a given epoch:

```
for epoch in range(5):
    print(epoch)
    train_epoch_losses, train_epoch_accuracies = [], []
```

3. Loop through batches of training data, and calculate the accuracy (`train_epoch_accuracy`) and loss value (`train_epoch_loss`) within an epoch:

```
for ix, batch in enumerate(iter(trn_dl)):
    x, y = batch
    batch_loss = train_batch(x, y, model, optimizer, loss_fn)
    train_epoch_losses.append(batch_loss)
train_epoch_loss = np.array(train_epoch_losses).mean()
for ix, batch in enumerate(iter(trn_dl)):
    x, y = batch
    is_correct = accuracy(x, y, model)
    train_epoch_accuracies.extend(is_correct)
train_epoch_accuracy = np.mean(train_epoch_accuracies)
```

4. Calculate the loss value and accuracy within the one batch of validation data (since the batch size of the validation data is equal to the length of the validation data):

```
for ix, batch in enumerate(iter(val_dl)):
    x, y = batch
    val_is_correct = accuracy(x, y, model)
    validation_loss = val_loss(x, y, model)
val_epoch_accuracy = np.mean(val_is_correct)
```

Note that in the preceding code, the loss value of the validation data is calculated using the `val_loss` function and is stored in the `validation_loss` variable. Furthermore, the accuracy of all the validation data points is stored in the `val_is_correct` list, while the mean of this is stored in the `val_epoch_accuracy` variable.

5. Finally, we append the training and validation datasets' accuracy and loss values to the lists that contain the epoch-level aggregate

validation and accuracy values. We're doing this so that we can look at the epoch level's improvement in the next step:

```
train_losses.append(train_epoch_loss)
train_accuracies.append(train_epoch_accuracy)
val_losses.append(validation_loss)
val_accuracies.append(val_epoch_accuracy)
```

9. Visualize the improvements in the accuracy and loss values in the training and validation datasets over increasing epochs:

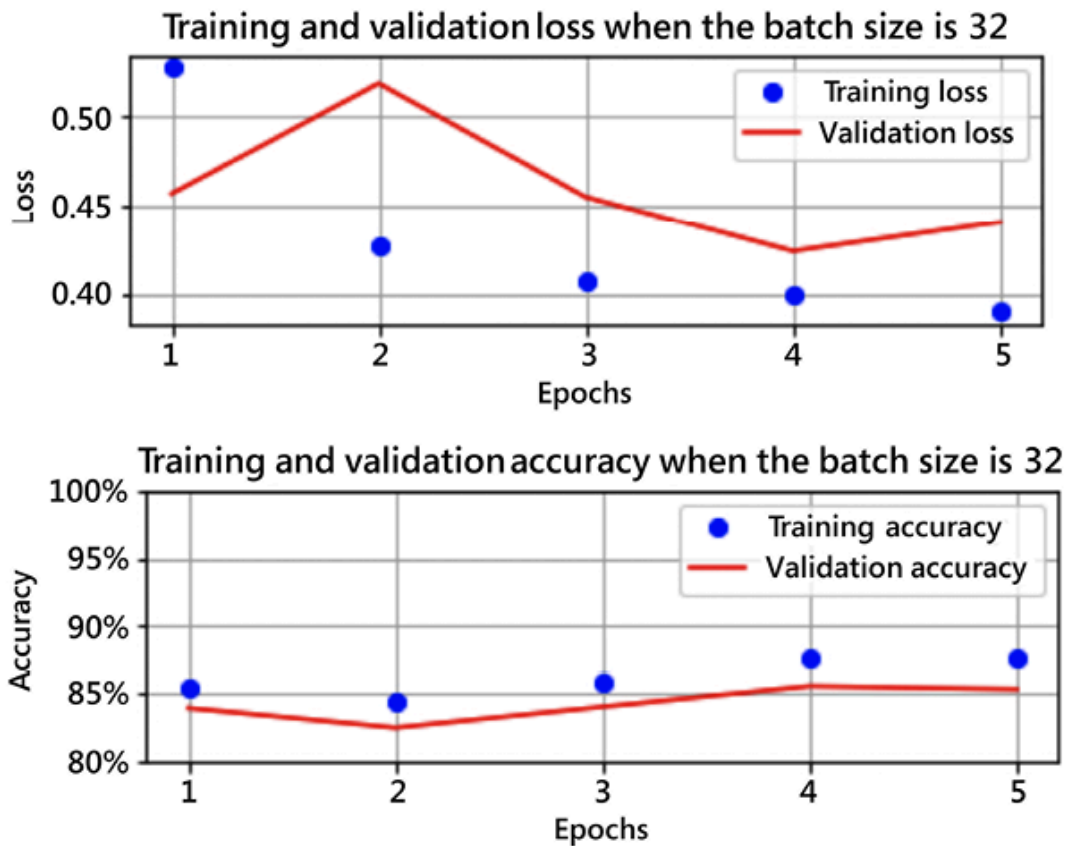


Figure 3.14: Training and validation loss and accuracy over increasing epochs with a batch size of 32

As you can see, the training and validation accuracy is ~85% by the end of five epochs when the batch size is 32. Next, we will vary the `batch_size` parameter when training the DataLoader in the `get_data` function to see its impact on accuracy at the end of five epochs.

Batch size of 10,000

In this section, we'll use 10,000 data points per batch so that we can understand what impact varying the batch size has.



Note that the code provided in the *Batch size of 32* section remains exactly the same here, except for the code in *step 5*, where we will specify a batch size of 10,000. We encourage you to refer to the respective notebook that's available in this book's GitHub repository while executing the code.

By making only this necessary change in *step 5* and after executing all the steps until *step 9*, the variation in the training and validation's accuracy and loss over increasing epochs when the batch size is 10,000 is as follows:

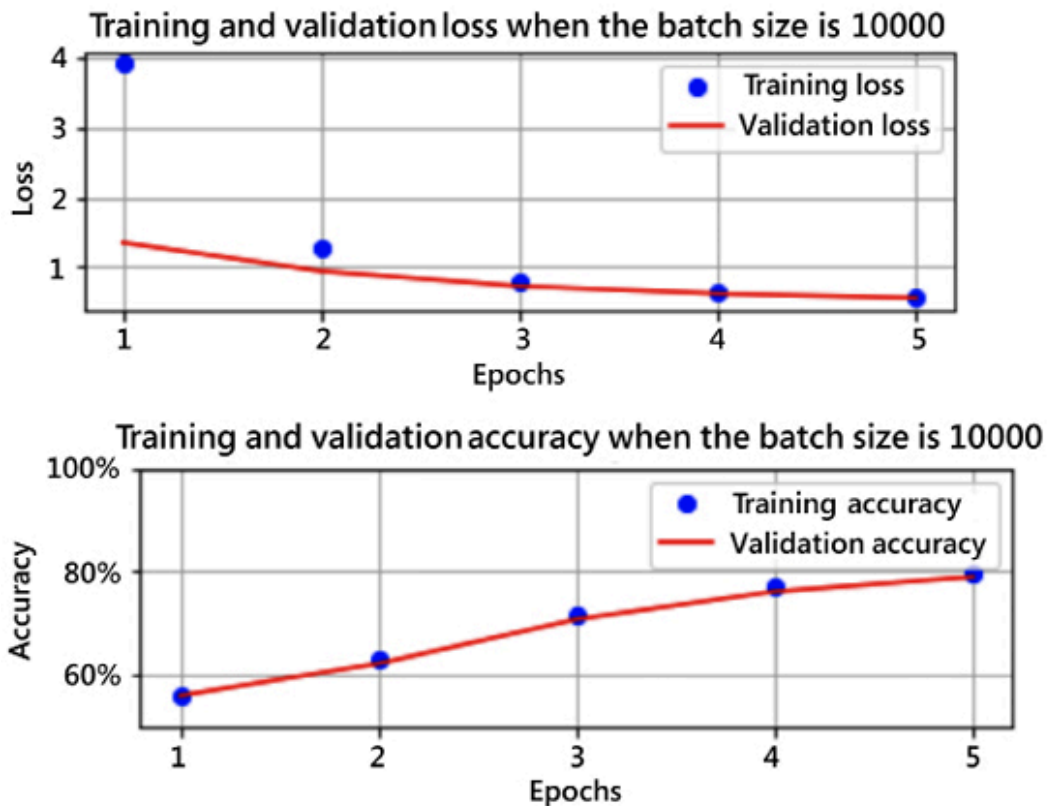


Figure 3.15: Training and validation loss and accuracy over increasing epochs with a batch size of 10,000

Here, we can see that the accuracy and loss values did not reach the same levels as that of the previous scenario, where the batch size was 32, because weights are updated fewer times per epoch (6 times) when compared to a batch size of 32 (1,875).



Having a lower batch size generally helps in achieving optimal accuracy when you have a small number of epochs, but it should not be so low that training time is impacted.

So far, we have learned how to scale a dataset, as well as the impact of varying the batch size on the model's training time to achieve a certain accuracy. In the next section, we will learn about the impact of varying the loss optimizer on the same dataset.

Understanding the impact of varying the loss optimizer

So far, we have optimized loss based on the Adam optimizer. A loss optimizer helps to arrive at optimal weight values to minimize overall loss. There are a variety of loss optimizers (different ways of updating weight values to minimize loss values) that impact the overall loss and accuracy of a model. In this section, we will do the following:

1. Modify the optimizer so that it becomes a **Stochastic Gradient Descent (SGD)** optimizer
2. Revert to a batch size of 32 while fetching data in the DataLoader
3. Increase the number of epochs to 10 (so that we can compare the performance of SGD and Adam over a longer number of epochs)

Making these changes means that only one step in the *Batch size of 32* section will change (since the batch size is already 32 in that section); that is, we will modify the optimizer so that it's the SGD optimizer.

Let's modify the `get_model` function in *step 4* of the *Batch size of 32* section to modify the optimizer so that we use the SGD optimizer instead, as follows:



The complete code can be found in the `Varying_loss_optimizer.ipynb` file located in the `Chapter03` folder on GitHub at <https://bit.ly/mcyp-2e>. For the sake of brevity, we will not detail every step from the *Batch size of 32* section; instead, in the following code, we will discuss only those steps where changes are introduced. **We encourage you to refer to the respective notebooks on GitHub while executing the code.**

1. Modify the optimizer so that you use the SGD optimizer in the `get_model` function while ensuring that everything else remains the same:

```
from torch.optim import SGD, Adam
def get_model():
    model = nn.Sequential(
        nn.Linear(28 * 28, 1000),
        nn.ReLU(),
        nn.Linear(1000, 10)
    ).to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = SGD(model.parameters(), lr=1e-2)
    return model, loss_fn, optimizer
```

Now, let's increase the number of epochs in *step 8* while keeping every other step (except for *steps 4* and *8*) the same as they are in the *Batch size of 32* section.

2. Increase the number of epochs we'll use to train the model.

After making these changes, once we execute all the remaining steps in the *Batch size of 32* section in order, the variation in the training and validation datasets' accuracy and loss values over increasing epochs (when trained with the SGD and Adam optimizers individually) will be as follows:

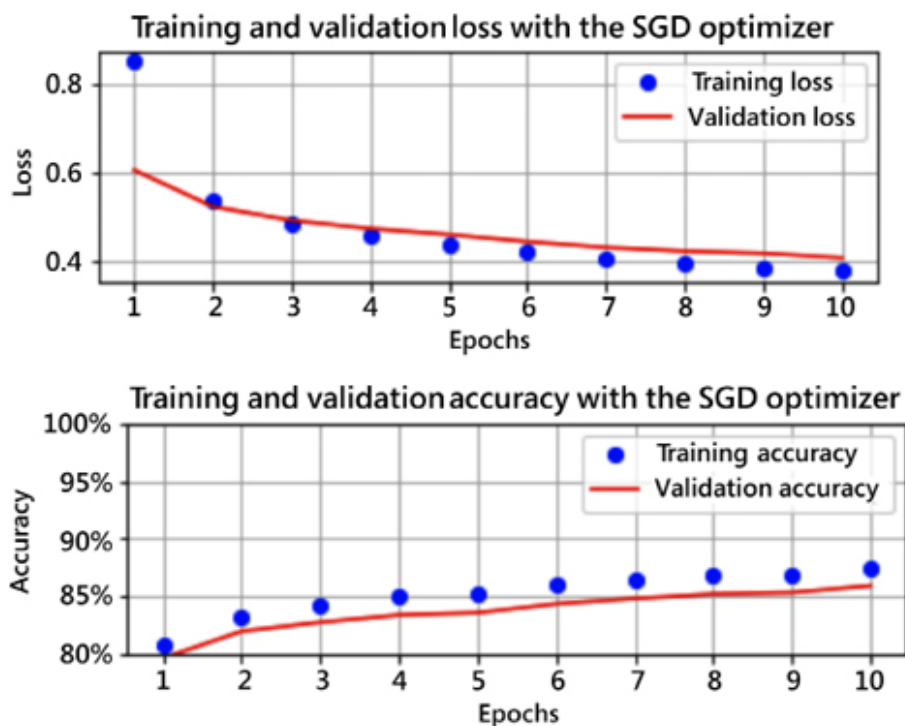


Figure 3.16: Training and validation loss and accuracy over increasing epochs with SGD optimizer

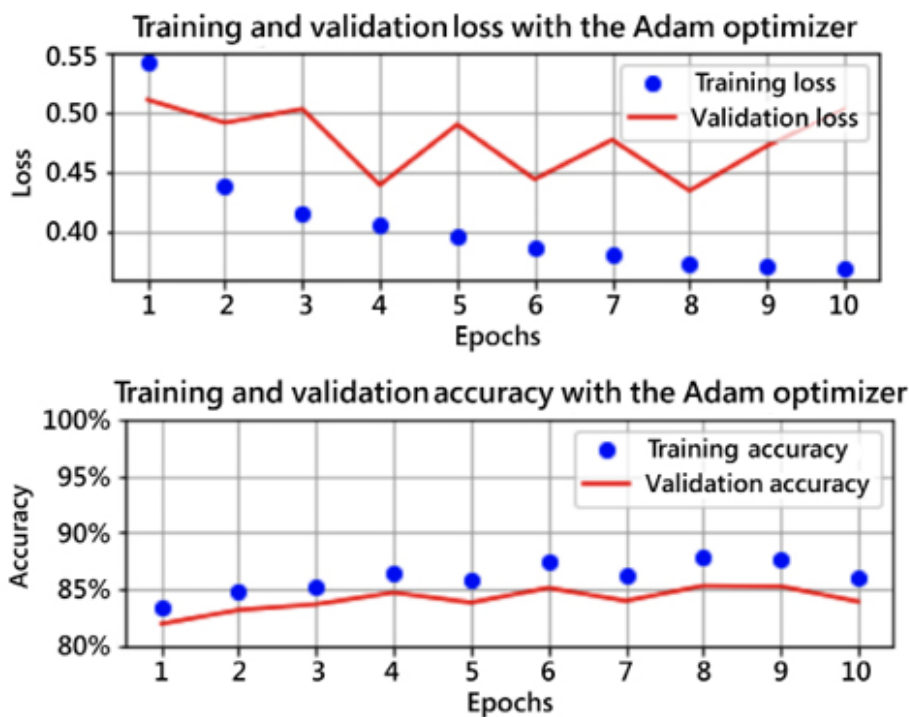


Figure 3.17: Training and validation loss and accuracy over increasing epochs with Adam optimizer

As you can see, when we used the Adam optimizer, the accuracy was still very close to 85%.



Note

Certain optimizers achieve optimal accuracy faster compared to others. Some of the other prominent optimizers that are available include Adagrad, Adadelata, AdamW, LBFGS, and RMSprop.

So far, we have used a learning rate of 0.01 while training our models and maintained it across all the epochs while training the model. In *Chapter 1, Artificial Neural Network Fundamentals*, we learned that the learning rate plays a key role in attaining optimal weight values. Here, the weight values gradually move toward the optimal value when the learning rate is small, while the weight values oscillate at a non-optimal value when the learning rate is large.

However, initially, it would be intuitive for the weights to be updated quickly to a near-optimal scenario. From then on, they should be updated very slowly, since the amount of loss that gets reduced initially is high and the amount of loss that gets reduced in the later epochs would be low.

This calls for having a high learning rate initially and gradually lowering it later as the model achieves near-optimal accuracy. This requires us to understand when the learning rate must be reduced (learning rate annealing over time). Refer to the `Learning_rate_annealing.ipynb` file located in the `Chapter03` folder on GitHub to understand the impact of learning rate annealing.

To understand the impact of the varying learning rate, the following scenarios will help:



- Higher learning rate (0.1) on a scaled dataset
- Lower learning rate (0.00001) on a scaled dataset
- Lower learning rate (0.00001) on a non-scaled dataset

These three scenarios will not be covered in this chapter; however, you can access the full code for them in the `Varying_learning_rate_on_scaled_data.ipynb` file and `Varying_learning_rate_on_non_scaled_data.ipynb` file in the `Chapter03` folder in the GitHub repository at <https://bit.ly/mcnp-2e>.

In the next section, we will learn about how the number of layers in a neural network impacts its accuracy.

Building a deeper neural network

So far, our neural network architecture only has one hidden layer. In this section, we will contrast the performance of models where there are two hidden layers and no hidden layer (with no hidden layer being a logistic regression).

A model with two layers within a network can be built as follows (note that we have kept the number of units in the second hidden layer set to 1,000). The modified `get_model` function (from the code in the *Batch size of 32* section), where there are two hidden layers, is as follows:



The full code can be found in the `Impact_of_building_a_deeper_neural_network.ipynb` file located in the `Chapter03` folder on GitHub at <https://bit.ly/mcnp-2e>. For the sake of brevity, we will not detail every step from the *Batch size of 32* section. Please refer to the notebooks on GitHub while executing the code.

```
def get_model():  
    model = nn.Sequential(  

```

```
        nn.Linear(28 * 28, 1000),
        nn.ReLU(),
        nn.Linear(1000, 1000),
        nn.ReLU(),
        nn.Linear(1000, 10)
    ).to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = Adam(model.parameters(), lr=1e-3)
    return model, loss_fn, optimizer
```

Similarly, the `get_model` function, where there are *no hidden layers*, is as follows:

```
def get_model():
    model = nn.Sequential(
        nn.Linear(28 * 28, 10)
    ).to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = Adam(model.parameters(), lr=1e-3)
    return model, loss_fn, optimizer
```

Note that, in the preceding function, we connect the input directly to the output layer.

Once we train the models as we did in the *Batch size of 32* section, the accuracy and loss on the train and validation datasets will be as follows:

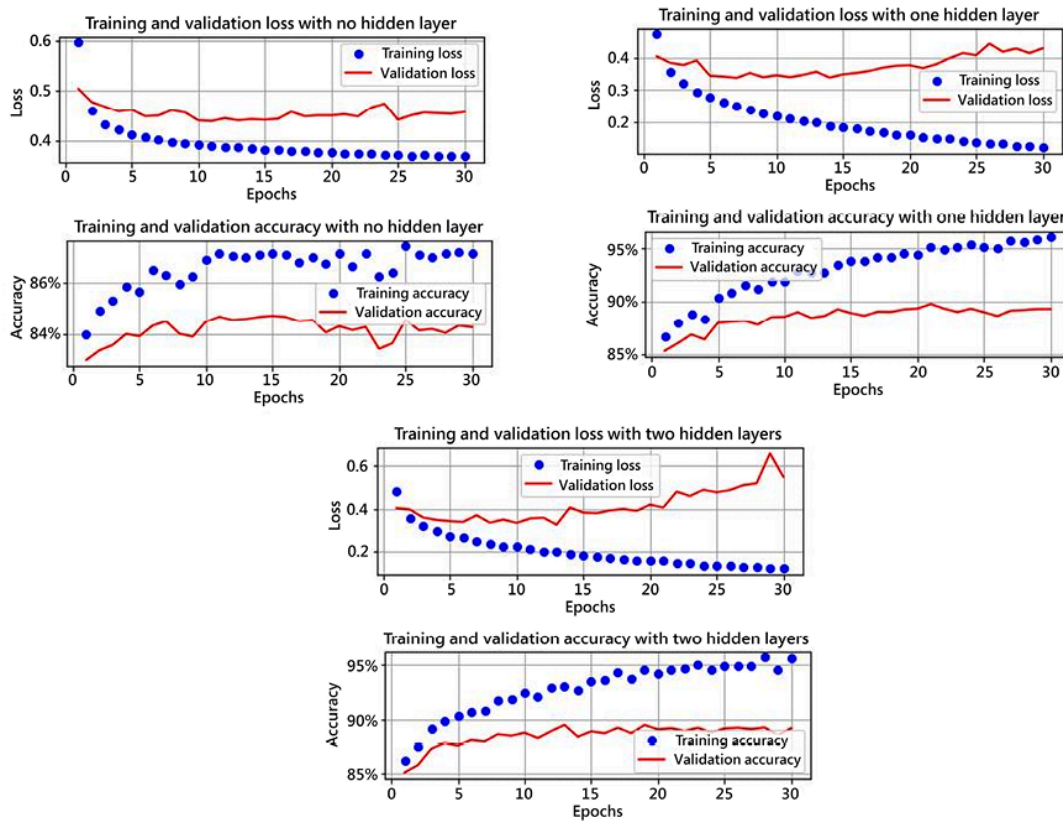


Figure 3.18: Training and validation loss and accuracy over increasing epochs with varying number of hidden layers

Here, take note of the following:

- The model was unable to learn well when there were no hidden layers.
- The model overfit by a larger amount when there were two hidden layers compared to one hidden layer (the validation loss is higher in the model with two layers compared to the model with one layer).

So far, across different sections, we have seen that the model was unable to be trained well when the input data wasn't scaled (brought down to a small range). Non-scaled data (data with a higher range) can also occur in hidden layers (especially when we have deep neural networks with multiple hidden layers) because of the matrix multiplication that's involved in getting the values of nodes in hidden layers. Let's learn about how batch normalization helps to deal with non-scaled data in the next section.

Understanding the impact of batch normalization

Previously, we learned that when the input value is large, the variation of the sigmoid output doesn't make much difference when the weight values change considerably.

Now, let's consider the opposite scenario, where the input values are very small:

Input	Weight	Sigmoid
0.01	0.00001	0.500
0.01	0.0001	0.500
0.01	0.001	0.500
0.01	0.01	0.500
0.01	0.1	0.500
0.01	0.2	0.500
0.01	0.3	0.501
0.01	0.4	0.501
0.01	0.5	0.501
0.01	0.6	0.501
0.01	0.7	0.502
0.01	0.8	0.502
0.01	0.9	0.502
0.01	1	0.502

Figure 3.19: Sigmoid value for the different values of input and weight

When the input value is very small, the sigmoid output changes slightly, requiring a big change to the weight value to achieve optimal results.

Additionally, in the *Scaling the input data* section, we saw that large input values have a negative effect on training accuracy. This suggests that we can neither have very small nor very big values for our input.

Along with very small or very big values in input, we may also encounter a scenario where the value of one of the nodes in the hidden layer could result in either a very small number or a very large number, resulting in the same issue we saw previously with the weights connecting the hidden layer to the next layer. Batch normalization comes to the rescue in such a scenario, since it normalizes the values at each node, just like when we scaled our input values. Typically, all the input values in a batch are scaled as follows:

$$\text{Batch mean } \mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

$$\text{Batch variance } \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\text{Normalized input } \bar{x}_i = \frac{(x_i - \mu_B)}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$\text{Batch normalized input} = \gamma \bar{x}_i + \beta$$

By subtracting each data point from the batch mean and then dividing it by the batch variance, we have normalized all the data points of the batch at a node to a fixed range. While this is known as hard normalization, by introducing the γ and β parameters, we let the network identify the best normalization parameters.

To understand how the batch normalization process helps, let's look at the loss and accuracy values on the training and validation datasets, as well as the distribution of hidden layer values, in the following scenarios:

1. Very small input values without batch normalization
2. Very small input values with batch normalization

Let's get started!

Very small input values without batch normalization

So far, when we had to scale input data, we scaled it at a value between 0 and 1. In this section, we will scale it further at a value between 0 and 0.0001 so that we can understand the impact of scaling data. As we saw at the beginning of this section, small input values cannot change the sigmoid value, even with a big variation in weight values.

To scale the input dataset so that it has a very low value, we'll change the scaling that we typically do in the `FMNISTDataset` class by reducing the range of input values from 0 to 0.0001, as follows:



The full code can be found in the `Batch_normalization.ipynb` file in the `Chapter03` folder on GitHub at <https://bit.ly/mcvp-2e>. For brevity, we will not detail every step from the *Batch size of 32* section. **Refer to the notebooks on GitHub while executing the code.**

```
class FMNISTDataset(Dataset):
    def __init__(self, x, y):
        x = x.float()/(255*10000) # Done only for us to
        # understand the impact of Batch normalization
        x = x.view(-1,28*28)
        self.x, self.y = x, y
    def __getitem__(self, ix):
        x, y = self.x[ix], self.y[ix]
        return x.to(device), y.to(device)
    def __len__(self):
        return len(self.x)
```

Note that in the lines of code we've highlighted (`x = x.float()/(255*10000)`), we have reduced the range of input pixel values by dividing them by 10,000.

Next, we must redefine the `get_model` function so that we can fetch the model's prediction, as well as the values for the hidden layer. We can do this by specifying a neural network class, as follows:

```
def get_model():
    class neuralnet(nn.Module):
        def __init__(self):
            super().__init__()
            self.input_to_hidden_layer = nn.Linear(784,1000)
            self.hidden_layer_activation = nn.ReLU()
            self.hidden_to_output_layer = nn.Linear(1000,10)
        def forward(self, x):
            x = self.input_to_hidden_layer(x)
            x1 = self.hidden_layer_activation(x)
            x2= self.hidden_to_output_layer(x1)
            return x2, x1
    model = neuralnet().to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = Adam(model.parameters(), lr=1e-3)
    return model, loss_fn, optimizer
```

In the preceding code, we defined the `neuralnet` class, which returns the output layer values (`x2`) and the hidden layer's activation values (`x1`). Note that the architecture of the network hasn't changed.

Given that the `get_model` function returns two outputs now, we need to modify the `train_batch` and `val_loss` functions, which make predictions, by passing input through the model. Here, we'll only fetch the output layer values, not the hidden layer values. Given that the output layer values are in the 0th index of what is returned from the model, we'll modify the functions so that they only fetch the 0th index of predictions, as follows:

```
def train_batch(x, y, model, opt, loss_fn):
    model.train()
    prediction = model(x)[0]
    batch_loss = loss_fn(prediction, y)
    batch_loss.backward()
```

```

optimizer.step()
optimizer.zero_grad()
return batch_loss.item()
def accuracy(x, y, model):
    model.eval()
    with torch.no_grad():
        prediction = model(x)[0]
    max_values, argmaxes = prediction.max(-1)
    is_correct = argmaxes == y
    return is_correct.cpu().numpy().tolist()

```

Note that the highlighted portions of the preceding code are where we have ensured we only fetch the 0th index of the model's output (since the 0th index contains the output layer's values).

Now, when we run the rest of the code provided in the *Scaling the data* section, we'll see that the variation in the accuracy and loss values in the training and validation datasets over increasing epochs is as follows:

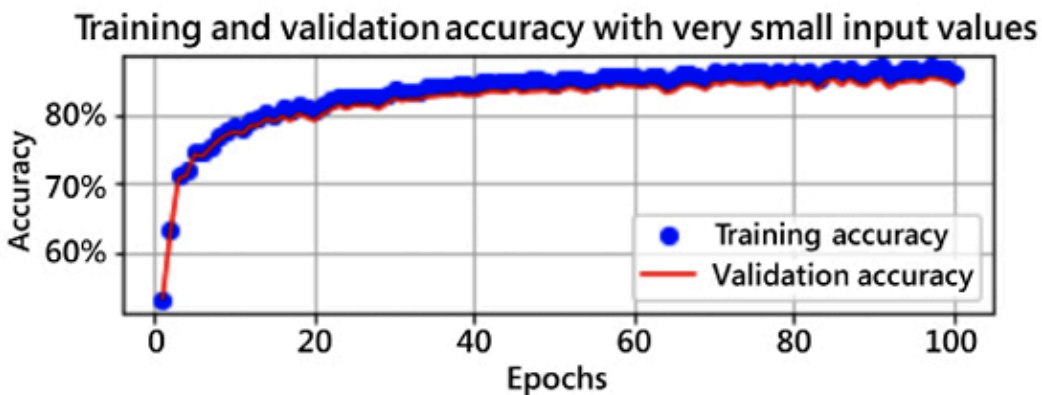
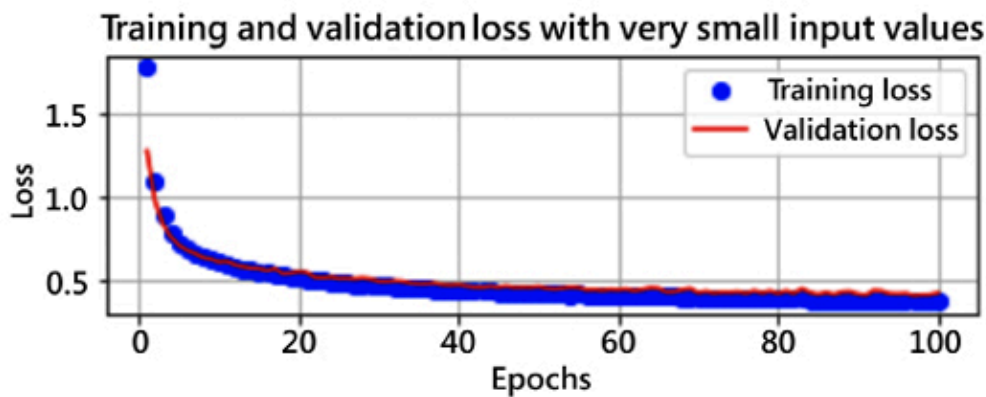


Figure 3.20: Training and validation loss and accuracy when network is trained with very small input values

Note that in the preceding scenario, the model didn't train well, even after 100 epochs (the model was trained to an accuracy of ~90% on the validation dataset within 10 epochs in the previous sections, while the current model only has ~85% validation accuracy).

Let's understand the reason why the model doesn't train as well when the input values have a very small range by exploring the hidden values' distribution, as well as the parameter distribution:

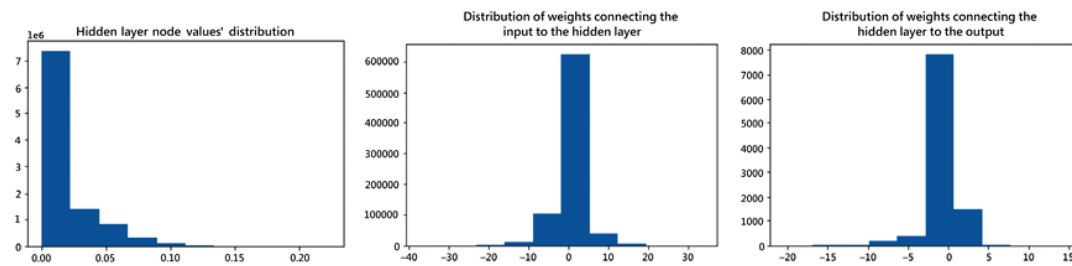


Figure 3.21: Distribution of weights and hidden layer node values when network is trained with very small input values

Note that the first distribution indicates the distribution of values in the hidden layer (where we can see that the values have a very small range). Furthermore, given that both the input and hidden layer values have a very small range, the weights had to be varied by a large amount (for both the weights that connect the input to the hidden layer and the weights that connect the hidden layer to the output layer).

Now that we know that the network doesn't train well when the input values have a very small range, let's understand how batch normalization helps increase the range of values within the hidden layer.

Very small input values with batch normalization

In this section, we'll only make one change to the code from the previous subsection; that is, we'll add batch normalization while defining the

model architecture.

The modified `get_model` function is as follows:

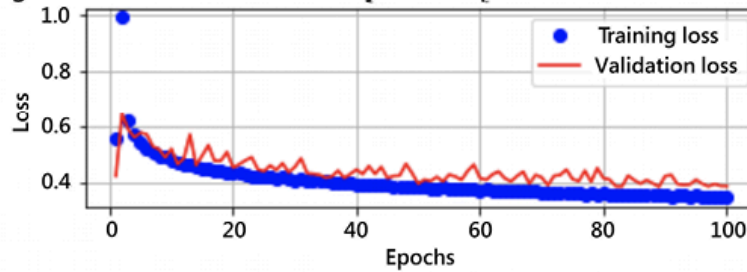
```
def get_model():
    class neuralnet(nn.Module):
        def __init__(self):
            super().__init__()
            self.input_to_hidden_layer = nn.Linear(784,1000)
            self.batch_norm = nn.BatchNorm1d(1000)
            self.hidden_layer_activation = nn.ReLU()
            self.hidden_to_output_layer = nn.Linear(1000,10)
        def forward(self, x):
            x = self.input_to_hidden_layer(x)
            x0 = self.batch_norm(x)
            x1 = self.hidden_layer_activation(x0)
            x2= self.hidden_to_output_layer(x1)
            return x2, x1
    model = neuralnet().to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = Adam(model.parameters(), lr=1e-3)
    return model, loss_fn, optimizer
```

In the preceding code, we declared a variable (`batch_norm`) that performs batch normalization (`nn.BatchNorm1d`). The reason we perform `nn.BatchNorm1d(1000)` is because the output dimension is 1,000 for each image (that is, a 1-dimensional output for the hidden layer).

Furthermore, in the `forward` method, we pass the output of the hidden layer values through batch normalization, prior to ReLU activation.

The variation in the training and validation datasets' accuracy and loss values over increasing epochs is as follows:

Training and validation loss with very small input values and batch normalization



Training and validation accuracy with very small input values and batch normalization

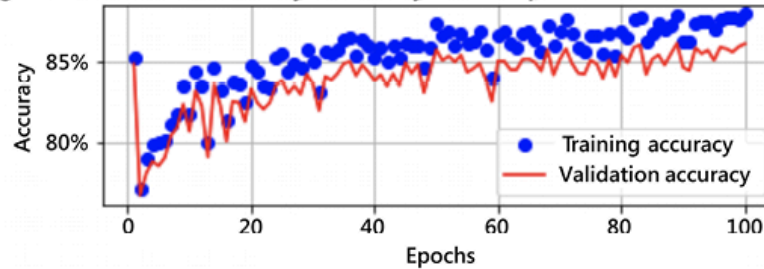


Figure 3.22: Training and validation loss when network is trained with very small input values and batch normalization

Here, we can see that the model was trained in a manner very similar to how it was trained when the input values did not have a very small range. Now, let's understand the distribution of hidden layer values and the weight distribution, as seen in the previous section:

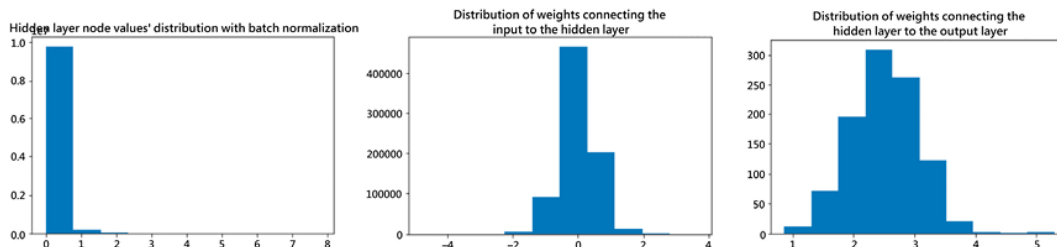


Figure 3.23: Distribution of weights and hidden layer node values when network is trained with very small input values and batch normalization

We can see that the hidden layer values have a wider spread when we have batch normalization and that the weights connecting the hidden layer to the output layer have a smaller distribution. This results in the model learning as effectively as it could in the previous sections.



Batch normalization helps considerably when training deep neural networks. It helps us avoid gradients becoming so small that the weights are barely updated.

So far, we have seen scenarios where training loss and accuracy are much better when compared to validation accuracy and loss: indicating a model that overly fits on training data but does not generalize well on validation datasets. We will look at fixing overfitting issues next.

The concept of overfitting

So far, we've seen that the accuracy of the training dataset is typically more than 95%, while the accuracy of the validation dataset is ~89%. Essentially, this indicates that a model does not generalize as much on unseen datasets, since it can learn from the training dataset. This also indicates that the model learns all the possible edge cases for the training dataset; these can't be applied to the validation dataset.



Having high accuracy on the training dataset and considerably lower accuracy on the validation dataset refers to the scenario of overfitting.

Some of the typical strategies that are employed to reduce the effect of overfitting are dropout and regularization. We will look at what impact they have on training and validation losses in the following sections.

Impact of adding dropout

We have already learned that whenever `loss.backward()` is calculated, a weight update happens. Typically, we would have hundreds of thousands of parameters within a network and thousands of data points to train our model on. This gives us the possibility that while most parameters help to train the model reasonably, certain parameters can be fine-tuned for the training images, resulting in their values being dictated by only a few images in the training dataset. This, in turn, results in the

training data having a high accuracy but not necessarily on the validation dataset.

Dropout is a mechanism that randomly chooses a specified percentage of node activations and reduces them to 0. In the next iteration, another random set of hidden units is switched off. This way, the neural network does not optimize for edge cases, as the network does not get that many opportunities to adjust the weight to memorize for edge cases (given that the weight is not updated in each iteration).

Keep in mind that, during prediction, dropout doesn't need to be applied, since this mechanism can only be applied while training a model.

Usually, there are cases where the layers behave differently during training and validation, as you saw in the case of dropout. For this reason, you must specify the mode for the model upfront using one of two methods:

`model.train()` to let the model know it is in training mode and `model.eval()` to let it know that it is in evaluation mode. If we don't do this, we might get unexpected results. For example, in the following image, notice how the model (which contains dropout) gives us different predictions on the same input when in training mode.

However, when the same model is in `eval` mode, it will suppress the dropout layer and return the same output:

```
[20]  1  model.train()
      2  batch = next(iter(trn_dl))
      3  for i in range(5):
      4      output = model(batch[0])
      5      print(output.mean().item())
```

```
[-13.275323867797852
-12.834677696228027
-11.895054817199707
-12.713885307312012
-13.302783012390137]
```

```
1  model.eval()
2  batch = next(iter(trn_dl))
3  for i in range(5):
4      output = model(batch[0])
5      print(output.mean().item())
```

```
[-12.40117359161377
-12.40117359161377
-12.40117359161377
-12.40117359161377
-12.40117359161377]
```

Figure 3.24: Impact of `model.eval()` on output values

While defining the architecture, `Dropout` is specified in the `get_model` function as follows:



The full code can be found in the `Impact_of_dropout.ipynb` file in the `Chapter03` folder on GitHub at <https://bit.ly/mcvp-2e>. For brevity, we will not detail every step from the *Batch size of 32* section. **Refer to the notebooks on GitHub while executing the code.**

```
def get_model():
    model = nn.Sequential(
        nn.Dropout(0.25),
        nn.Linear(28 * 28, 1000),
```

```

        nn.ReLU(),
        nn.Dropout(0.25),
        nn.Linear(1000, 10)
    ).to(device)
    loss_fn = nn.CrossEntropyLoss()
    optimizer = Adam(model.parameters(), lr=1e-3)
    return model, loss_fn, optimizer

```

Note that, in the preceding code, `Dropout` is specified before linear activation. This specifies that a fixed percentage of the weights in the linear activation layer won't be updated.

Once the model training is completed, as in the *Batch size of 32* section, the loss and accuracy values of the training and validation datasets will be as follows:

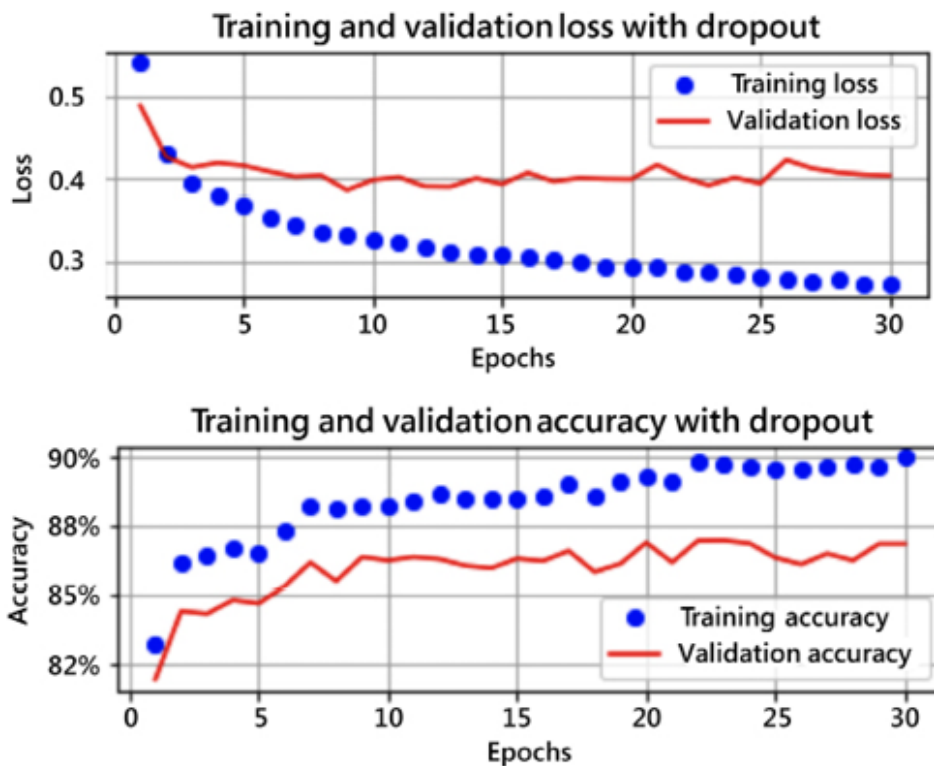


Figure 3.25: Training and validation loss and accuracy with dropout

In this scenario, the delta between the training and validation datasets' accuracy is not as large as we saw in the previous scenario, thus result-

ing in a scenario that has less overfitting.

Impact of regularization

Apart from the training accuracy being much higher than the validation accuracy, one other feature of overfitting is that certain weight values will be much higher than the other weight values. Large weight values can be a symptom of a model learning very well on training data (essentially, rote learning based on what it has seen).

While dropout is a mechanism that's used so that the weight values aren't updated as frequently, regularization is another mechanism that we can use for this purpose.

Regularization is a technique in which we penalize the model for having large weight values. Hence, it is an objective function that minimizes the loss of training data, as well as the weight values. In this section, we will learn about two types of regularization: L1 regularization and L2 regularization.



The full code can be found in the `Impact_of_regularization.ipynb` file in the `Chapter03` folder on GitHub at <https://bit.ly/mcvp-2e>. For brevity, we will not detail every step from the *Batch size of 32* section. **Refer to the notebooks on GitHub while executing the code.**

Let's get started!

L1 regularization

L1 regularization is calculated as follows:

$$L1 \text{ loss} = -\frac{1}{n} \left(\sum_{i=1}^n (y_i * \log(p_i) + (1 - y_i) * \log(1 - p_i)) \right) + \Delta \sum_{j=1}^m |w_j|$$

The first part of the preceding formula refers to the categorical cross-entropy loss that we have used for optimization so far, while the second part refers to the absolute sum of the weight values of the model. Note that L1 regularization ensures that it penalizes for the high absolute values of weights by incorporating them in the loss value calculation.

Δ refers to the weightage that we associate with the regularization (weight minimization) loss.

L1 regularization is implemented while training the model, as follows:

```
def train_batch(x, y, model, opt, loss_fn):
    model.train()
    prediction = model(x)
    l1_regularization = 0
    for param in model.parameters():
        l1_regularization += torch.norm(param, 1)
    batch_loss = loss_fn(prediction, y) + 0.0001 * l1_regularization
    batch_loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    return batch_loss.item()
```

As you can see, we have enforced regularization on the weights and biases across all the layers by initializing `l1_regularization`.

`torch.norm(param, 1)` provides the absolute value of the weight and bias values across layers. Furthermore, we have a very small weightage (`0.0001`) associated with the sum of the absolute value of the parameters across layers.

Once we execute the remaining code, as in the *Batch size of 32* section, the training and validation datasets' loss and accuracy values over increasing epochs will be as follows:

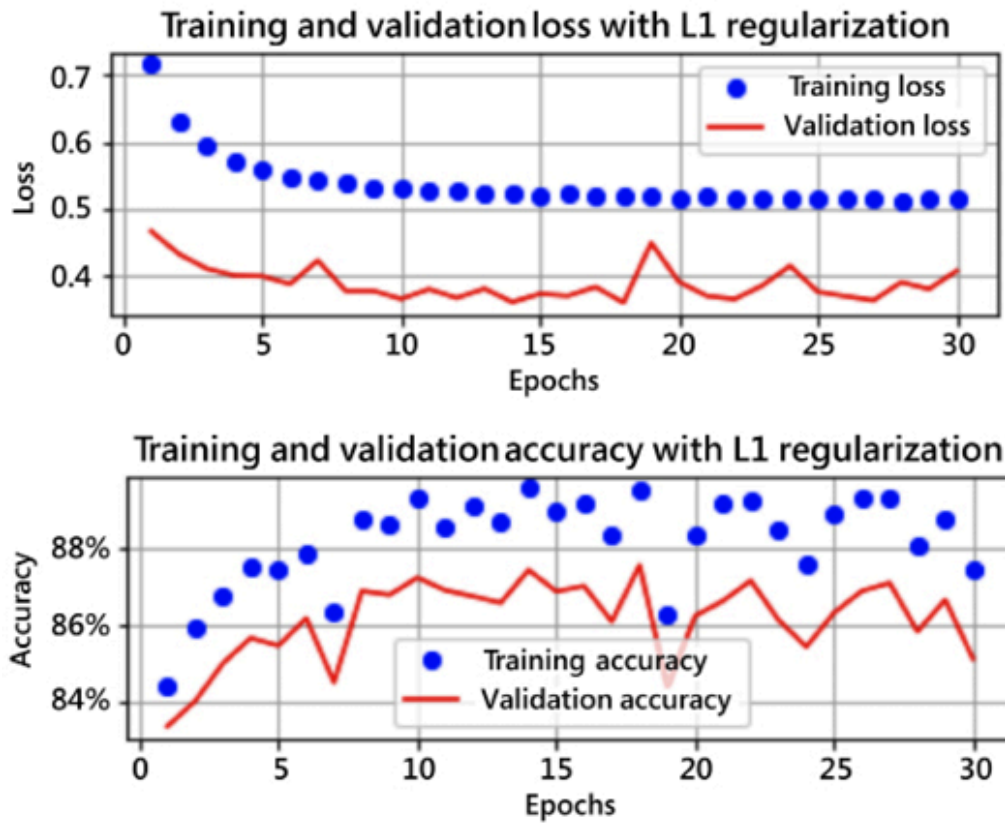


Figure 3.26: Training and validation loss and accuracy with L1 regularization

As you can see, the difference between the training and validation datasets' accuracy is not as high as it was without L1 regularization.

L2 regularization

L2 regularization is calculated as follows:

$$L2 \text{ loss} = -\frac{1}{n} \left(\sum_{i=1}^n (y_i * \log(p_i) + (1 - y_i) * \log(1 - p_i)) \right) + \Delta \sum_{j=1}^m w_j^2$$

The first part of the preceding formula refers to the categorical cross-entropy loss obtained, while the second part refers to the squared sum of the weight values of the model. Similar to L1 regularization, we penalize for large weight values by having the sum of squared values of weights incorporated into the loss value calculation.

Δ refers to the weightage that we associate with the regularization (weight minimization) loss.

L2 regularization is implemented while training the model, as follows:

```
def train_batch(x, y, model, opt, loss_fn):
    model.train()
    prediction = model(x)
    l2_regularization = 0
    for param in model.parameters():
        l2_regularization += torch.norm(param, 2)
    batch_loss = loss_fn(prediction, y) + 0.01*l2_regularization
    batch_loss.backward()
    optimizer.step()
    optimizer.zero_grad()
    return batch_loss.item()
```

In the preceding code, the regularization parameter, Δ (0.01), is slightly higher than in L1 regularization, since the weights are generally between -1 and 1, and squaring them would result in even smaller values.

Multiplying them by an even smaller number, as we did in L1 regularization, would result in us having very little weightage for regularization in the overall loss calculation. Once we execute the remaining code, as in the *Batch size of 32* section, the training and validation datasets' loss and accuracy values over increasing epochs will be as follows:

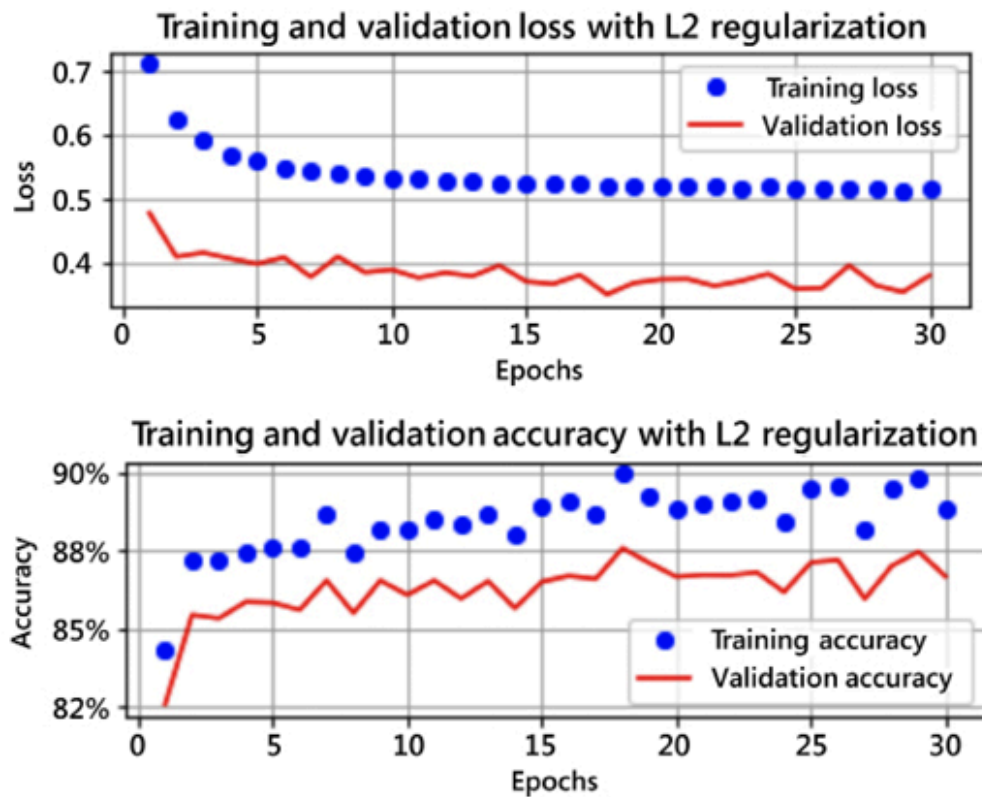


Figure 3.27: Training and validation loss and accuracy with L2 regularization

We can see that L2 regularization has also resulted in the validation and training datasets' accuracy and loss values being closer to each other.

Summary

We started this chapter by learning about how an image is represented. Then, we learned about how scaling, the value of the learning rate, our choice of optimizer, and the batch size help improve the accuracy and speed of training. We then learned about how batch normalization helps to increase the speed of training and addresses the issues of very small or large values in a hidden layer. Then, we learned about scheduling the learning rate to increase accuracy further. We then proceeded to understand the concept of overfitting and learned about how dropout and L1 and L2 regularization help us avoid overfitting.

Now that we have learned about image classification using a deep neural network, as well as the various hyperparameters that help train a model,

in the next chapter, we will learn about how what we've learned in this chapter can fail and how to address this, using convolutional neural networks.

Questions

1. What happens if the input values are not scaled in the input dataset?
2. What could happen if the background has a white pixel color while the content has a black pixel color when training a neural network?
3. What is the impact of batch size on a model's training time and memory?
4. What is the impact of the input value range have on weight distribution at the end of training?
5. How does batch normalization help improve accuracy?
6. Why do weights behave differently during training and evaluation in the dropout layer?
7. How do we know if a model has overfitted on training data?
8. How does regularization help in avoiding overfitting?
9. How do L1 and L2 regularization differ from each other?
10. How does dropout help in reducing overfitting?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>



